Implementation of Trend Sales Prediction

Bachelor's Thesis in Mathematical Engineering

Carlo Siniscalchi

August 2025

Contents

Abstract

Abstract in lingua italiana

0.1 Introduction

0.1.1 The requested result

The prose theater "Piccolo Teatro" had the need to integrate a forecasting model into a Power BI dashboard to analyze show ticket purchase trends. In particular as a final result they wanted a plot showing 2 trend lines, the first calculated using only the data of the the first N days after the sales of the show are opened. Also this line is constrained to have the finale point as their sales target but by preserving the original shape. The second line instead will be calculated using all the data known.

0.1.2 Purpose

This project was requested by the marketing team of the theater, their idea is to compare the two trend lines to decide if they need to invest or not in marketing to reach their sales goal. The constrained line would represent their target, and because the shape is preserved they can decide based if they have to invest accordingly to the fact the the real prediction is under or over the target line.

0.2 Data Pipeline

First of all it is possible to find all the project code in this GitHub repository All the code I will talk about in this section is inside `/piccolo_teatro/data_pipeline`

0.2.1 Data Presentation

`D_CONFIG_PROD_LIST`: This table contains information about the theater shows. Each row is a transaction made by the theater.

`D_SALES_LIST_SALES`: This table contains information about each transaction at the theater. Each row is a performance of a show.

`stagioni`: This table contains the start and end dates of the theater seasons. This data is particularly relevant because the opening of ticket sales for theater shows coincides with the beginning of the season, while the end of sales corresponds to the day of the performance.

0.2.2 Data Ingestion

In the `/data_ingestion.py` file I defined 3 classes (Sales, Products, Seasons) one for each table, this way it's easier to manage and clean data. Indeed each class takes as input a Pandas Dataframe and has specific methods to mandage the Dataframe. For Example inside the Sales class we have the following methods:

- **rename_cols**: Renames columns to easily usable names, removing any leading spaces.
- **assign_types**: Standardizes the decimal separator in the `gain` column (sometimes ",", other times "."), drops rows with missing values, and converts data types (float, int, datetime, str).

- **clean_cols**: Reduces the DataFrame to only the columns necessary for analysis.
- **clean_rows**: Removes unwanted seasons and filters to only ticket-sale transactions (e.g., excludes subscription purchases).
- **assign_index**: Sets the `date` column as the DataFrame index.
- **sort_values**: Sorts the DataFrame chronologically by `date`.
- **get_head(*offset*)**: Keeps only the initial fraction of rows defined by *offset*, simulating the first N days of sales.
- **get_same_show(*show_id*)**: Filters the DataFrame for a specific show.
- **get_groups**: Returns a `GroupBy` object grouped by `show_id` for aggregate analysis.
- **clean**: Runs `assign_types`, `clean_rows`, and `clean_cols` in sequence for a full cleaning pipeline.

Dataframes after ingestion:

- **Sales DF:**
  Shape: `553171 rows × 8 cols`

Column	Type
individuali_gruppi	object
online_offline	object
gain	float64
tickets	int64
date	datetime64[ns]
season_id	int64
show_id	int64
performance_id	int64

- **Shows DF:**
  Shape: `7355 rows × 14 cols`

Column	Type
season_name	object
D_CONFIG_PROD_LIST_T_SEASON_ID	int64
show_id	int64
D_CONFIG_PROD_LIST_PRODUCT_STATE	object
D_CONFIG_PROD_LIST_T_PERFORMANCE_ID	int64
performance_day	object
space	object
D_CONFIG_PROD_LIST_T_SPACE_ID	float64
D_CONFIG_PROD_LIST_LOG_CONFIG	object
D_CONFIG_PROD_LIST_T_LOGCONFIG_ID	float64
performance_state	object
max_tickets	int64
D_CONFIG_PROD_LIST_PRODUCT_EXTERNAL_NAME	object
show_type	object

- **Seasons DF:**
  Shape: `11 rows × 6 cols`

Column	Type
season_id	int64
season_name	object
inizio_vendite	datetime64[ns]
fine_vendite	datetime64[ns]
inizio_stagione	datetime64[ns]
fine_stagione	datetime64[ns]

0.3 Features and Targets

0.3.1 Approach

Since feature calculation can be time-consuming and the final set of features and targets is not known in advance, we compute all potentially useful features from the raw data once and then use a configuration file to select the subset to feed into the model. We start with three separate cleaned tables that are not yet related: one containing show information (eg. type, number of performances), another containing transaction records, and a third listing seasons. Our goal is to merge these into a single table where each row represents a show on a specific day, enriched with both historical transaction metrics and static show metadata for model training.

0.3.2 Calculate Features and Targets

The code for this section can be found in `/piccolo_teatro/data_pipeline/transformation.py`.

First, we group the sales data by `show_id`, `season`, and `day`, summing all transactions for each show on each day of a season. For instance, each entry in the `gain` column represents the total revenue generated by a show on that specific day. From this aggregated data, we compute the following features:

- **Sales Metrics and Targets:**
  Features marked with * can serve as prediction targets when calculated one day after the data point.
  - ***gain_cum_sum**: cumulative daily revenue.
  - ***tickets_cum_sum**: cumulative daily ticket sales.
  - **{colName}_mvg_avg_{period}**: rolling-window moving averages over {period} days (periods: 2, 4, 6, 8, 10, 15, 20, 30) of `gain_cum_sum`, `tickets_cum_sum`, and `percentage_bought`.
    Justification: captures short-term versus medium-term momentum in ticket sales (e.g., whether demand is accelerating or decelerating). By smoothing out day-to-day noise, it helps models learn the underlying trend rather than spurious fluctuations.
  - **{colName}_shifted_{period}**: lagged values over {period} days of `gain_cum_sum`, `tickets_cum_sum`, and `percentage_bought` (e.g., the value of `gain_cum_sum` six days prior).

Justification: quantifies autocorrelation in sales: today's sales often depend on what happened 1, 3, or 7 days ago (weekend effects, promotional spikes). Including multiple lags lets the model learn optimal weights for each horizon.

- **remaining_tickets**: tickets still available.
  Justification: provides a "remaining capacity" signal. As tickets sell out, scarcity can drive a last-minute rush—critical to capture for near-sell-out shows.
- ***percentage_bought**: proportion of tickets sold to date relative to the total available.
  Justification: normalizes cumulative sales, helping the model compare shows of different sizes and highlighting stages of sell-through.

- **Show Metadata:**
  Static information about each show.
  - **show_type**: e.g., children's show, cultural collaboration.
    Justification: different genres have distinct sales patterns (a children's show may have steadier sales; famous shows often see rapid early sell-through).
  - **show_capacity**: total tickets available across all performances.
    Justification: overall scale influences pace of sales and potential for scarcity effects.
  - **num_performances**: total number of performances for the show.
    Justification: more performances spread out demand, often smoothing daily sales curves compared to single-night events.

- **Temporal Distances:**
  Measures of position within the sales calendar.
  - **start_sales_distance**: days since the start of ticket sales.
    Justification: marks the phase of the sales cycle (initial launch spike versus steady growth).
  - **end_season_distance**: days remaining until season close.
    Justification: proximity to season end can spur urgency across shows as the period of availability shrinks.
  - **end_sales_distance**: days remaining until the show's final performance.
    Justification: captures last-minute buying behavior as audiences scramble before the final date.
  - **sales_duration**: total days tickets are on sale.
    Justification: informs expected trend shape (long campaigns may plateau; short ones often accelerate).
  - **percentage_sales_day**: fraction of elapsed sales days relative to `sales_duration`.
    Justification: normalizes temporal position, allowing comparison across shows with different sale windows.

To illustrate typical behaviors, we plot examples of shows with long and short selling periods and those exhibiting a final sales peak. Across these, we observe roughly exponential growth patterns, motivating a logarithmic transformation of sales metrics before modeling.

Finally, we split the dataset into training, validation, and test sets. To prevent data leakage, all records for a given show are assigned to the same split.

0.3.3 Managing Features

We now have more features than we actually need to train our model. To keep things simple at first—and only add complexity when necessary—we require an easy way to enable or disable individual features. That's why, in `/piccolo_teatro/config/__init__.py`, we define our feature set declaratively using Pydantic.

Pydantic gives us a clean, type-safe, declarative syntax for configuration schemas, complete with automatic validation, sensible defaults, and informative error messages.

In order to flexibly enable or disable individual feature transformations and to support our iterative forecasting scheme (where a prediction at time $t + 1$ feeds into the feature set for predicting $t + 2$, and so on), we implement a small framework based on a `Feature` class. This framework encapsulates:

- Which raw columns each feature concerns,
- Whether the feature is constant (does not change over time) or variable,
- Whether it is currently enabled,
- A callable `update` method to compute its new value.

```
class Feature(BaseModel):
    columns: List[str]
    const: bool
    enabled: bool
    update: Callable = None
```

All features are collected in a `Features` container. Upon initialization, it builds three lists:

- `enabled_features`: all `Feature` instances with `enabled=True`,
- `const_features`: those in `enabled_features` with `const=True`,
- `variable_features`: those with `const=False`.

```
class Features:
    def __init__(self, df=None):
        self.df             = df
        self.new_prediction = None

        # Example of a constant feature:
        self.performance_capacity = Feature(
            columns=["performance_capacity"],
            const=True, enabled=False,
            update=self.__update_const
        )

        # Example of a variable feature:
        self.percentage_sales_day = Feature(
            columns=["percentage_sales_day"],
            const=False, enabled=True,
            update=self.__update_percentage_sales_day
        )

        # Build the lists of features:
        self.__get_features()

    def __get_features(self):
        """
        Get a list of all the features, constant features, variable
        ↪  features
        """
        self.enabled_features = []
        self.const_features = []
        self.variable_features = []
        for name, value in vars(self).items():
```

```
            if isinstance(value, Feature):
                self.enabled_features.append(value)
                if value.enabled:
                    if value.const:
                        self.const_features.append(value)
                    else:
                        self.variable_features.append(value)
```

After each time-step prediction, we call `update_features(prediction)`. It:

1. Stores the new prediction,
2. Calls each variable feature's `update` to compute its value for the next row,
3. Calls each constant feature's `update` (simply copying the last known value),
4. Concatenates the newly computed row to the data frame.

```
def update_features(self, prediction):
  self.new_prediction = prediction
  self.new_row = {}

  # Compute variable features
  for feature in self.variable_features:
      feature.update()

  # Copy constant features
  for feature in self.const_features:
      feature.update(feature.columns[0])

  # Append to DataFrame
  self.new_row = pd.DataFrame(self.new_row)
  self.df = pd.concat([self.df, self.new_row], ignore_index=True)
```

For example the function used to update `percentage_sales_day`:

```
def __update_percentage_sales_day(self):
  # Extract the last row of the existing DataFrame
  last_row = self.df.iloc[[-1]]
  # Increment the days since sales start
  increased_day = int(last_row["start_sales_distance"].iloc[0]) +
  ↪  1
  # Compute new percentage relative to full sales duration
```

```
        new_value = increased_day / self.df["sales_duration"].iloc[0]
        # Store it in the row-to-be-appended
        self.new_row["percentage_sales_day"] = [new_value]
```

In this update:

- We take the previous `start_sales_distance` and add 1 day,
- Divide by the constant `sales_duration` (retrieved from the first row),
- And save the result under `percentage_sales_day` for the new time step.

0.4 Prediction

spiegazione modelli candidati, come ho intenzione di prevedere, come voglio testare (metrica test)

0.5 Linear Regression

log transformation, vedere diagnostica che posso ho fatto in statistca, ridge??

0.5.1 Analisi esplorativa (!!inf!!)

- Statistiche descrittive di variabili numeriche e categoriali. - Grafici univariati (istogrammi, boxplot) per individuare distribuzioni e outlier. - Scatter-plot target vs ogni feature per capire relazioni lineari/non lineari. - Matrice di correlazione per verificare multicollinearità.

Trasformazioni: - Se feature o target molto asimmetrici: considera log, sqrt, box–cox. - Standardizza (Z-score) o normalizza (MinMax) sempre le feature numeriche per confronto dei coefficienti e stabilità numerica.

Selezione iniziale delle feature: - Rimuovi feature costanti (varianza = 0). - Rimuovi feature con correlazione molto alta (pairwise > 0.9) per limitare multicollinearità.

0.5.2 Verifica Assunzioni della Regressione Lineare

- Linearità - Indipendenza errori - Omogeneità della varianza - Normalità della varianza - Multicollinearità

0.5.3 Valutazione del modello

- Analisi coefficienti - Diagnostica residui

0.5.4 Regolarizzazione se necessario

- Ridge - Lasso - ElasticNet

0.5.5 Interpretazione e Reporting

- R squared - Significatività statistica - Feature importance relativa -

0.6 XGB

- Definizione dei parametri iniziali e Cross-validation con early stopping - Training - Feature importance - SHAP - Ottimizzazione avanzata Bayes, Hyperopt o Optuna per eta, max_depth, min_child_weight, colsample_*, subsample, lambda, alpha. - Schemi di sampling più sofisticati (sklearn wrappers, GPU boost)